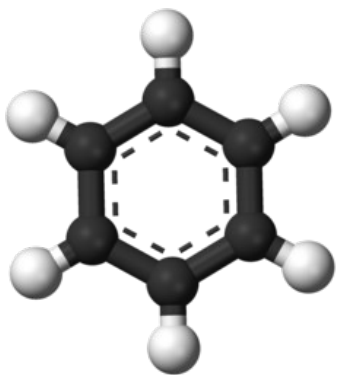


Практические задачи машинного обучения

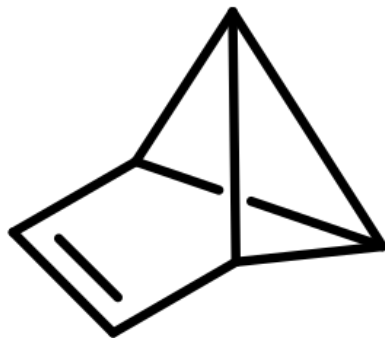
Бернгардт О.И.

Что за вещество C_6H_6 ?

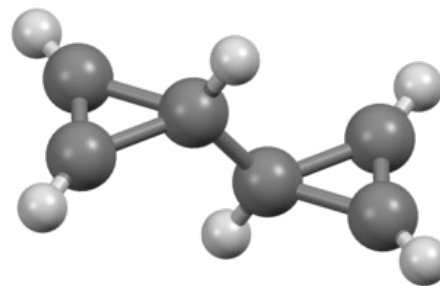
C_6H_6 - одна формула, много разных веществ



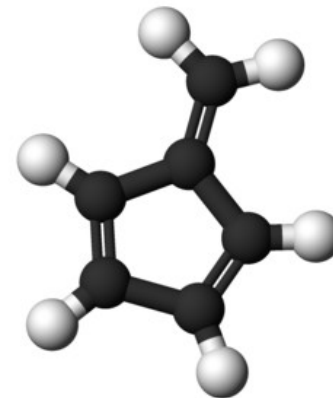
Бензол



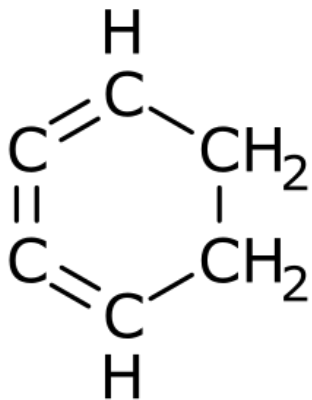
Бензвален



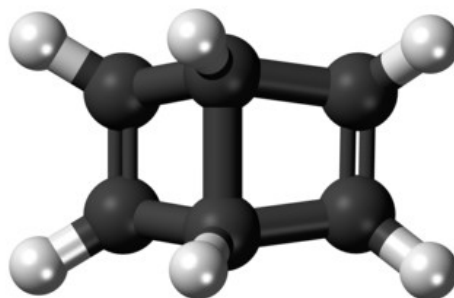
Бициклопропенил



Пентафальвен



Циклогексотриен



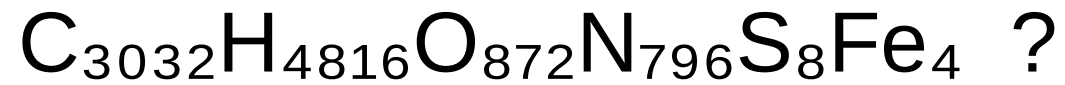
Деварбензен



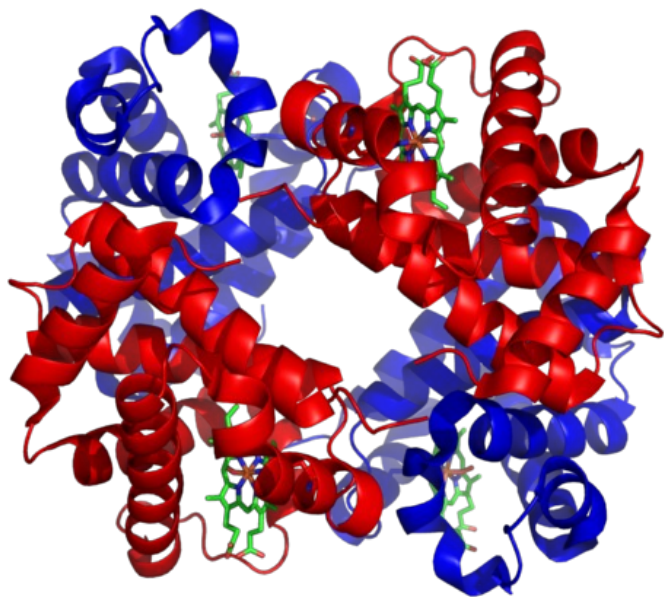
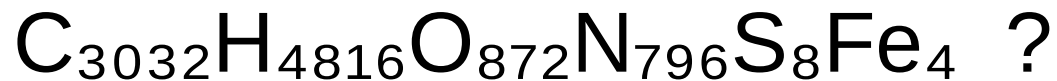
Призман

и это еще не все варианты...

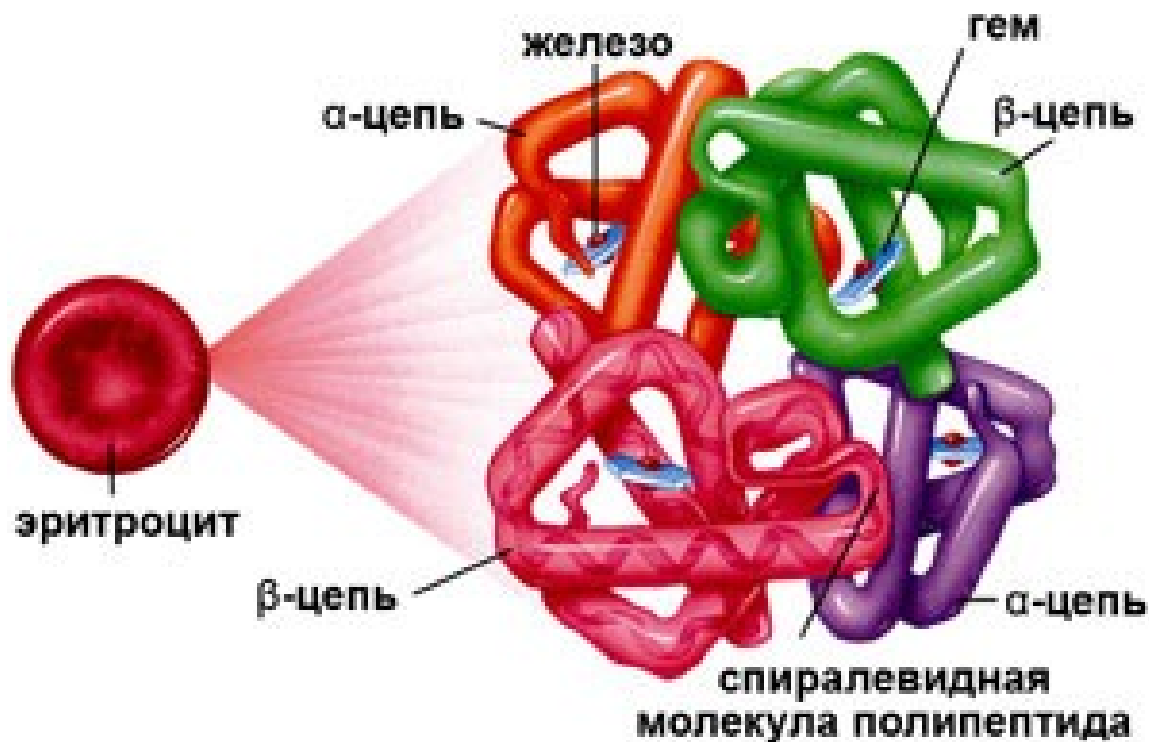
Сколько вариантов у вещества:



Сколько вариантов у вещества:



Гемоглобин



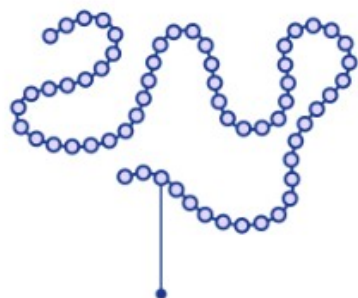
Проблема:

Можно-ли по формуле вещества и
реакциях в которых он участвует
определить его трехмерную структуру?

Восстановление трехмерной структуры белка нейросетевой моделью Alpha Fold

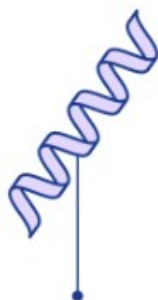
A protein chain acquires its native 3D structure

Every protein is made up of a sequence of amino acids bonded together

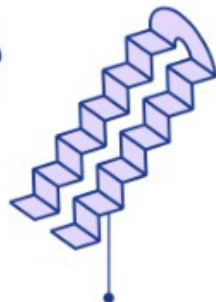


Amino acids

These amino acids interact locally to form shapes like helices and sheets



Alpha helix



Pleated sheet

These shapes fold up on larger scales to form the full three-dimensional protein structure



Pleated sheet

Alpha helix

Proteins can interact with other proteins, performing functions such as signalling and transcribing DNA



Восстановление трехмерной структуры белка нейросетевой моделью Alpha Fold



T1037 / 6vr4
90.7 GDT

(RNA polymerase domain)



T1049 / 6y4f
93.3 GDT

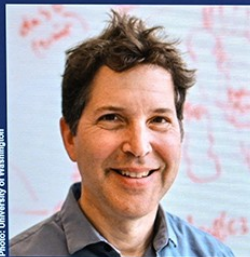
(adhesin tip)

Median Free-Modelling Accuracy



NOBELPRISET I KEMI 2024
THE NOBEL PRIZE IN CHEMISTRY 2024

KUNGL.
VETENSKAPS
AKADEMIEN
THE ROYAL SWEDISH ACADEMY OF SCIENCES



David Baker
University of Washington
USA

"för datorbaserad proteindesign"
"for computational protein design"



Demis Hassabis
Google DeepMind
United Kingdom

"för proteinstrukturprediktion"
"for protein structure prediction"



John M. Jumper
Google DeepMind
United Kingdom

Скачать программу:
<http://github.com/google-deepmind/alphafold>

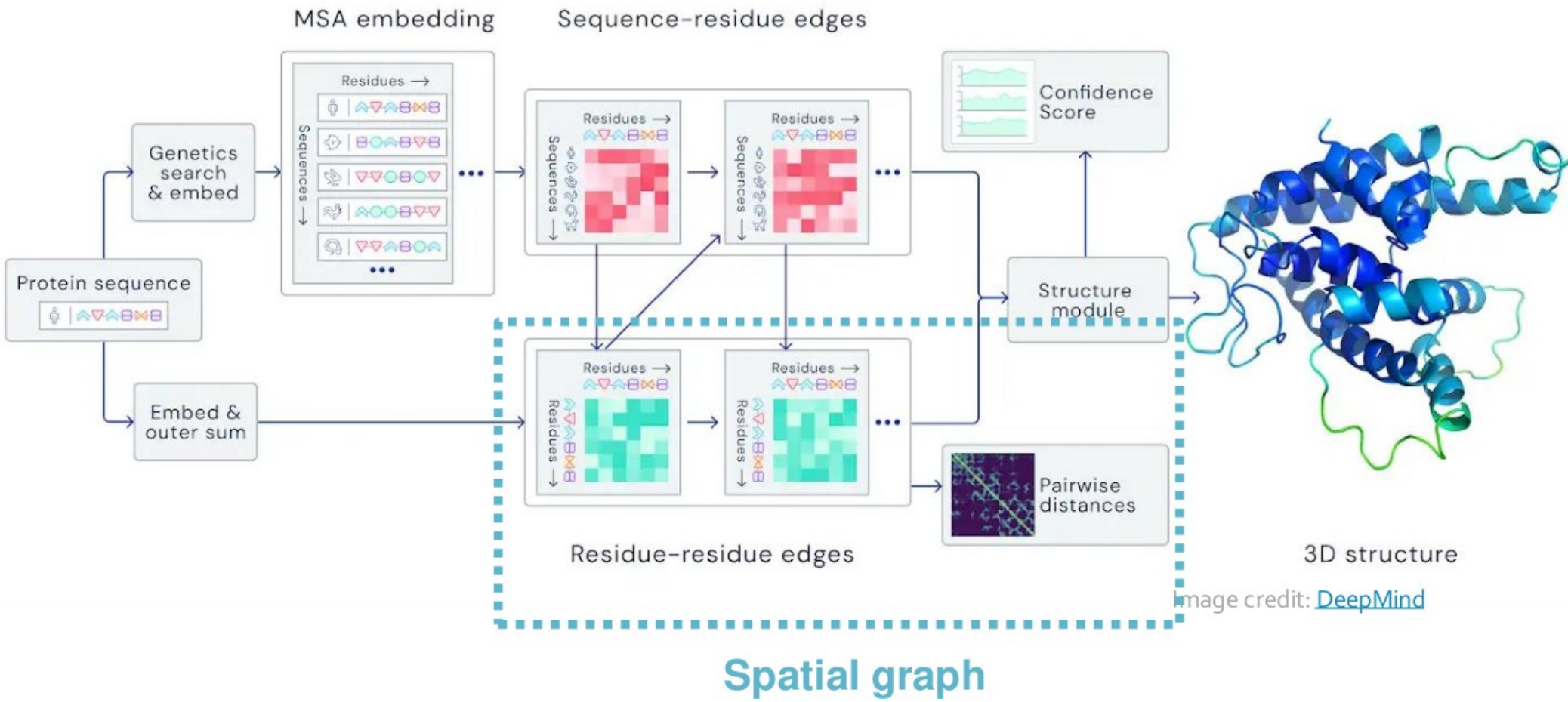


Image credit: [DeepMind](#)

Литература

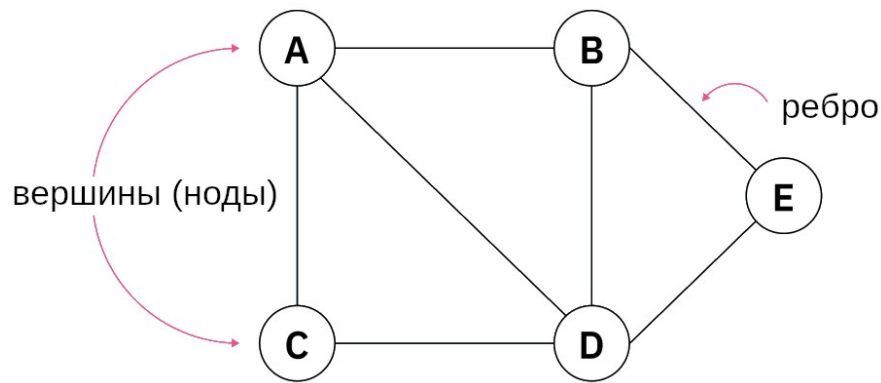
Основная

- Онлайн лекции J.Leskovec, Machine learning with graphs, Stanford, 2021, <https://www.youtube.com/watch?list=PLoROMvodv4rPLKxlpqhjhPgdy7imNkDn>
- M.Labonne, Hands-On Graph Neural Networks Using Python//Packt, 2023, 332pp.
- Y.Ma and J.Tang, Deep Learning on Graphs//Cambridge University Press, 2021, 400pp, https://yaoma24.github.io/dlg_book/
- L.Wu, P.Cui, J.Pei, L.Zhao, Graph Neural Networks: Foundations, Frontiers, and Applications//Springer, 2022, 689pp, <https://doi.org/10.1007/978-981-16-6054-2>
- K.Broadwater, N.Stillman, Graph Neural Networks in Action//Manning, 2025, 392pp.

Вспомогательная

- W.L. Hamilton, Graph Representation Learning// Synthesis Lectures on Artificial Intelligence and Machine Learning, 2020, Vol. 14, No. 3 , Pages 1-159.
- D.Easley,J.Kleinberg, Networks, Crowds, and Markets: Reasoning about a Highly Connected World//Cambridge University Press, 2010, 819pp,
- Z.Liu and J.Zhou, Introduction to Graph Neural Networks//Springer, 2020, 127pp, <https://doi.org/10.1007/978-3-031-01587-8>
- A.Negro, Graph-Powered Machine Learning// Manning, 2020, 467pp

Что такое граф?



Порядок графа — это число его вершин $|V|$.

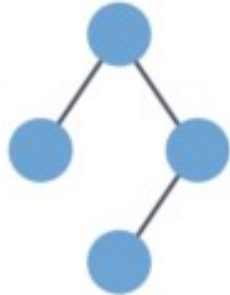
Размер графа — это число его рёбер $|E|$.

Степень вершины — это число смежных с ней рёбер. Соседи вершины v в графе G — это подмножество вершины V_i , образованное всеми вершинами, смежными с v .

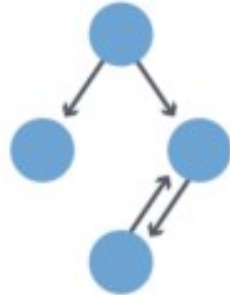
Граф соседства (также известный как эго-граф) вершины v в графе G — это подграф графа G , состоящий из вершин, смежных с v , и всех рёбер, соединяющих вершины, смежные с v .

Виды графов

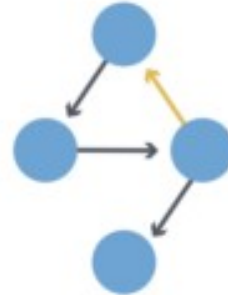
Undirected



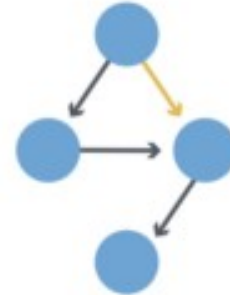
Directed



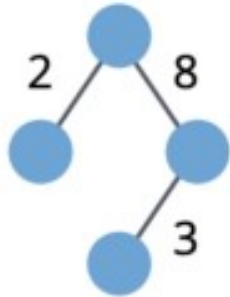
Cyclic



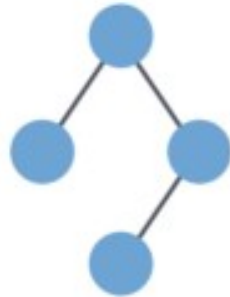
Acyclic



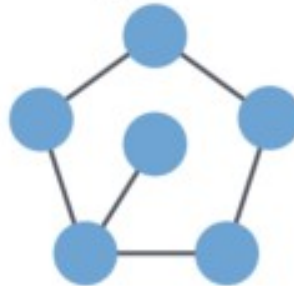
Weighted



Unweighted

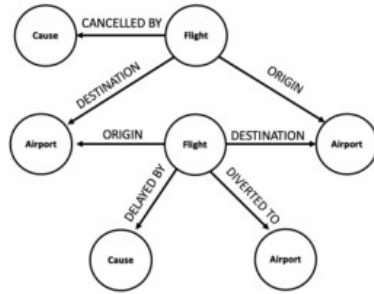


Sparse



Dense



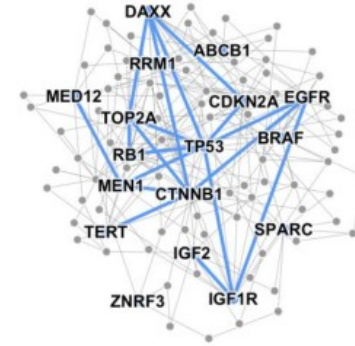


Event Graphs



Image credit: [SalientNetworks](#)

Computer Networks



Disease Pathways

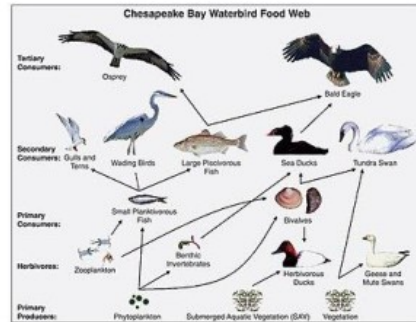


Image credit: [Wikipedia](#)

Food Webs



Image credit: [Pinterest](#)

Particle Networks



Image credit: [visitlondon.com](#)

Underground Networks

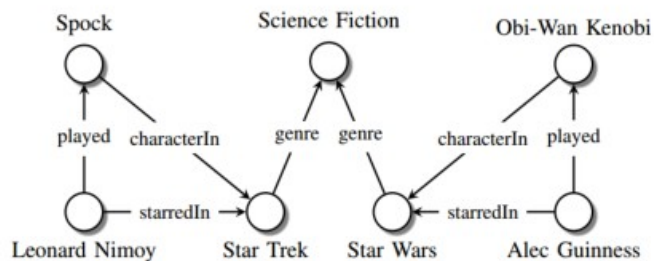


Image credit: [Maximilian Nickel et al](#)

Knowledge Graphs

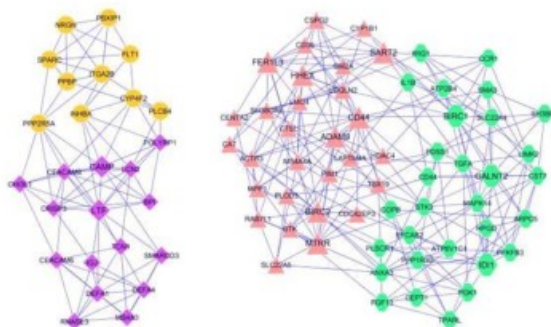


Image credit: [ese.wustl.edu](#)

Regulatory Networks

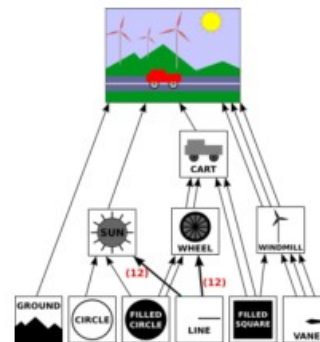


Image credit: [math.hws.edu](#)

Scene Graphs

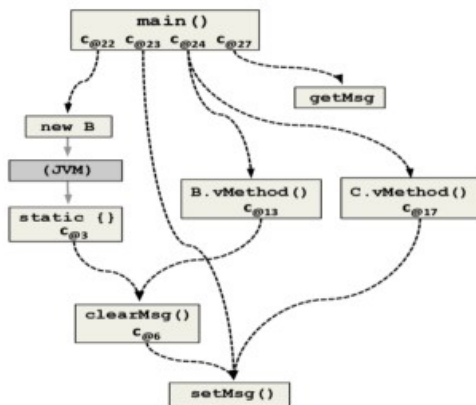


Image credit: [ResearchGate](#)

Code Graphs

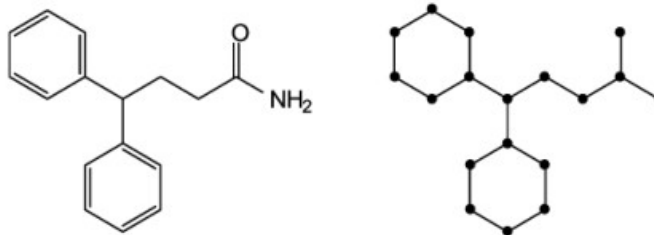


Image credit: [MDPI](#)

Molecules

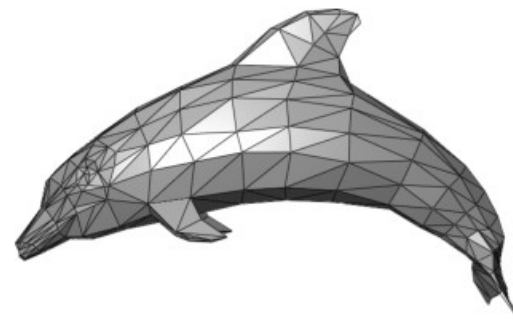


Image credit: [Wikipedia](#)

3D Shapes



Image credit: [Medium](#)

Social Networks

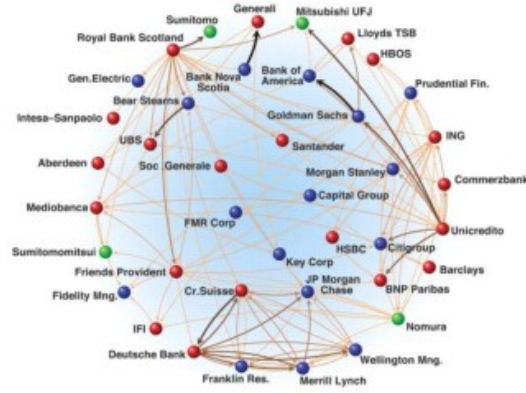


Image credit: [Science](#)

Economic Networks



Image credit: [Lumen Learning](#)

Communication Networks



Citation Networks



Image credit: [Missoula Current News](#)

Internet

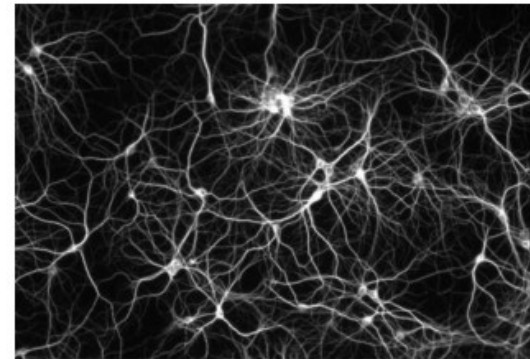
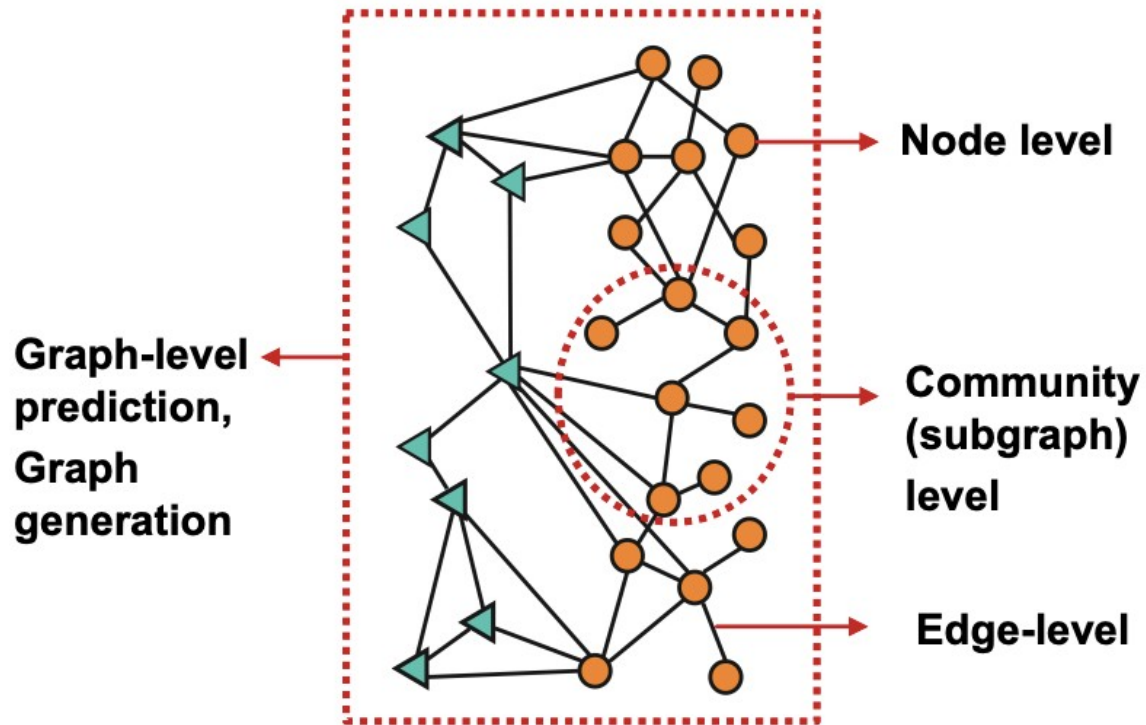


Image credit: [The Conversation](#)

Networks of Neurons

Основные решаемые задачи на графах



Генерация графов

Случайные графы. Модель Эрдёша — Реньи

- В модели $G(n, M)$ граф выбирается однородно и случайно из набора всех графов, которые имеют n вершин и M рёбер. Например, в модели $G(3, 2)$ каждый из трёх возможных графов с тремя вершинами и двумя рёбрами выбираются с вероятностью $1/3$.
- В модели $G(n, p)$ граф строится путём случайного добавления рёбер. Каждое ребро включается в граф с вероятностью p независимо от остальных рёбер. Эквивалентно, все графы с n узлами и M рёбрами имеют одинаковую вероятность.

$$p^M (1 - p)^{\binom{n}{2} - M}.$$

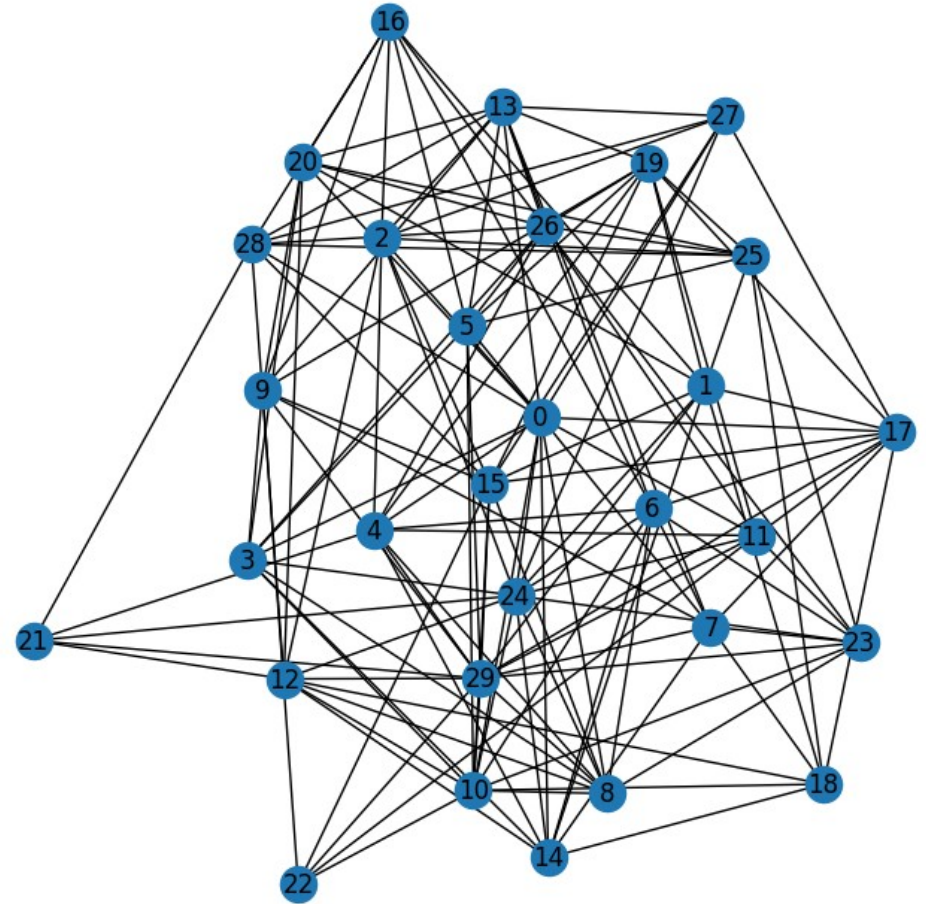
Параметр p в этой модели можно рассматривать как функцию веса. По мере роста p от 0 к 1 модель включает с большей вероятностью графы с большим числом рёбер. В частности, случай $p=0,5$ соответствует случаю, когда все $2^{\binom{n}{2}}$ графы с n вершинами будут выбраны с равной вероятностью.

Математическое ожидание числа рёбер в $G(n, p)$ равно $\binom{n}{2}p$ и по закону больших чисел любой граф в $G(n, p)$ будет почти наверняка иметь примерно это число рёбер. Поэтому, грубо говоря, если $pn^2 \rightarrow \infty$, то $G(n, p)$ должен вести себя подобно $G(n, M)$ с $M=\binom{n}{2}p$ при росте n .

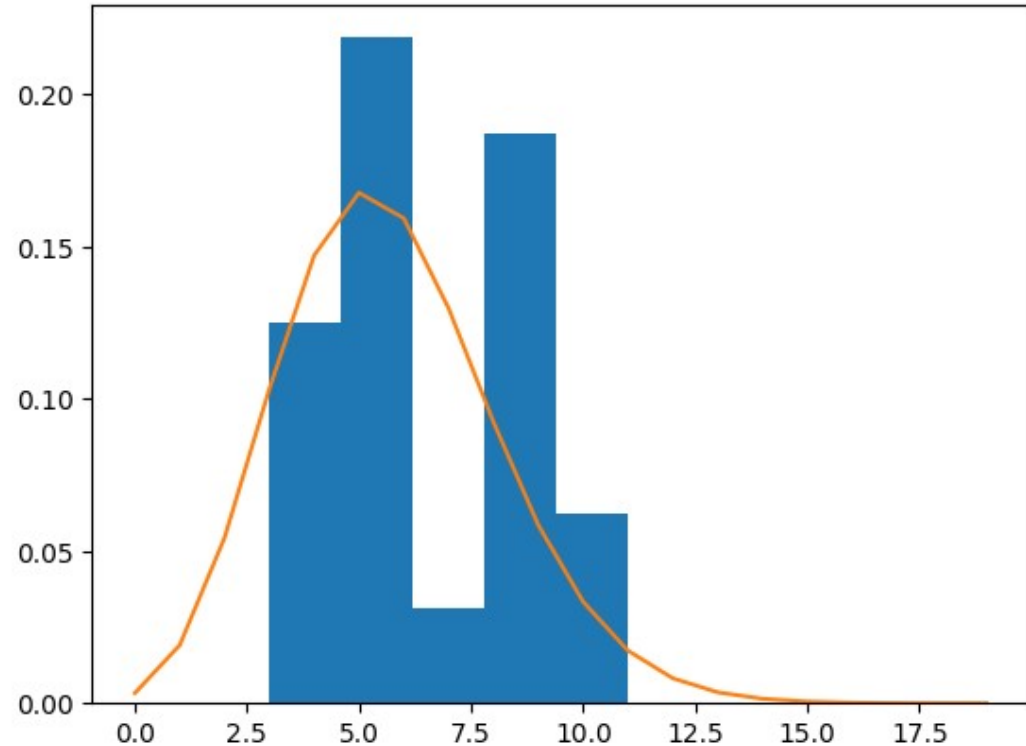
$$p_k = \binom{n}{k} p^k (1 - p)^{n-k}.$$

Асимптотически – это распределение Пуассона с $\lambda = (n-1)p$

```
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
N=30
P=0.3
G = nx.erdos_renyi_graph(N, P, seed=42)
plt.figure(figsize=(7, 7))
pos = nx.spring_layout(G, seed=42)
nx.draw(G, pos, with_labels=True)
```



```
num_nodes = G.number_of_nodes()
num_edges = G.number_of_edges()
avg_degree = sum(dict(G.degree()).values()) / num_nodes if num_nodes > 0 else 0
print("Nodes: ",num_nodes," Edges: ",num_edges,"Avg Degree: ",avg_degree)
degrees = [d for n, d in G.degree()]
plt.hist(degrees,bins=round(np.sqrt(N))+1,density=True)
from scipy.stats import poisson
dist=poisson((N-1)*P)
l=np.array(range(N))
plt.plot(l,dist.pmf(l))
```



Расстоянием d между вершинами v_i и v_j называется длина минимального пути между этими вершинами.

Диаметром δ связного графа G называется максимальное возможное расстояние между любыми двумя его вершинами.

Для несвязных графов диаметр полагается равным бесконечности.

Центром графа G называется такая вершина v , что максимальное расстояние между v и любой другой вершиной является наименьшим из всех возможных. Это расстояние называется радиусом r . Таким образом,

$$r = \min(\max d(v, v_2)) ,$$

где $d(v, v_2)$ – расстояние между v и v_2 .

Вычисление в библиотеке `networkx`:

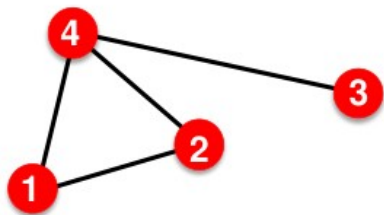
диаметр : `nx.diameter(G)`

радиус : `nx.radius(G)`

центр: `nx.center(G)`

Матрица смежности

Undirected



$$A_{ij} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

$$A_{ij} = A_{ji}$$

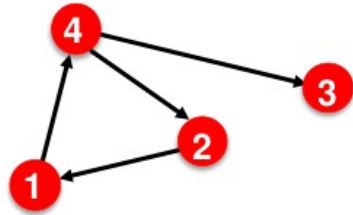
$$A_{ii} = 0$$

$$k_i = \sum_{j=1}^N A_{ij}$$

$$k_j = \sum_{i=1}^N A_{ij}$$

$$L = \frac{1}{2} \sum_{i=1}^N k_i = \frac{1}{2} \sum_{ij} A_{ij}$$

Directed



$$A = \begin{pmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{pmatrix}$$

$$A_{ij} \neq A_{ji}$$

$$A_{ii} = 0$$

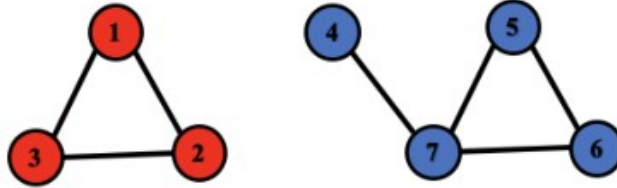
$$k_i^{out} = \sum_{j=1}^N A_{ij}$$

$$k_j^{in} = \sum_{i=1}^N A_{ij}$$

$$L = \sum_{i=1}^N k_i^{in} = \sum_{j=1}^N k_j^{out} = \sum_{i,j} A_{ij}$$

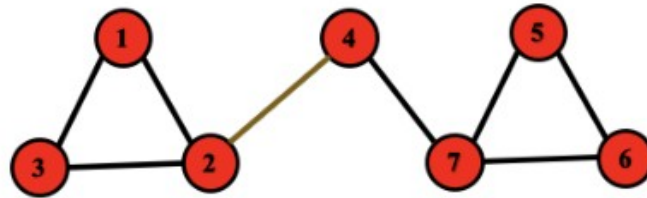
Связные и несвязные графы

Disconnected



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

Connected



$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 \end{pmatrix}$$

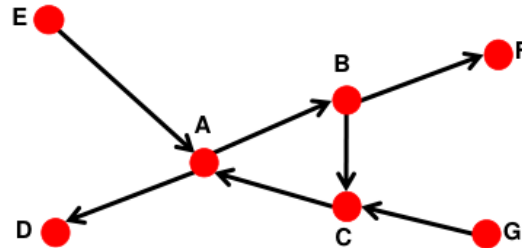
Связность направленных графов

- **Strongly connected directed graph**

- has a path from each node to every other node and vice versa (e.g., A-B path and B-A path)

- **Weakly connected directed graph**

- is connected if we disregard the edge directions



Graph on the left is connected but not strongly connected (e

Теория 6 рукопожатий

Теория шести рукопожатий — социологическая теория, согласно которой любые два человека на Земле разделены не более чем пятью уровнями общих знакомых (и, соответственно, шестью уровнями связей).

Формальная математическая формулировка теории — диаметр графа знакомств не превышает 6.

Случайные графы. Модель Барабаши — Альберт (модель предпочтительного соединения)

Сеть начинается с начальной сетки с m_0 узлами. $m_0 \geq 2$ и степень каждого узла в начальной сети должна быть не меньше 1, иначе она всегда будет отделена от остальной части сети.

В каждый момент времени в сеть добавляется новый узел. Каждый новый узел соединяется с существующими узлами с вероятностью, пропорциональной числу связей этих узлов. Формально, вероятностью p_i того, что новый узел соединится с узлом i равна:^[1]

$$p_i = \frac{k_i}{\sum_j k_j}$$

где k_i — степень i -го узла, а в знаменателе суммируются степени всех существующих узлов. Наиболее связанные узлы («хабы»), как

$$P(k) \sim k^{-3}$$

Сети со степенным законом распределения степеней узлов называют безмасштабными.

Принцип предпочтительно присоединения — пример положительной обратной связи, где изначально случайные вариации (один узел изначально имеет больше ссылок или начинает собирать ссылки раньше других) автоматически усиливаются, тем самым значительно увеличивая разрыв. Это также иногда называют эффектом Матфея, «богатые становятся богаче»

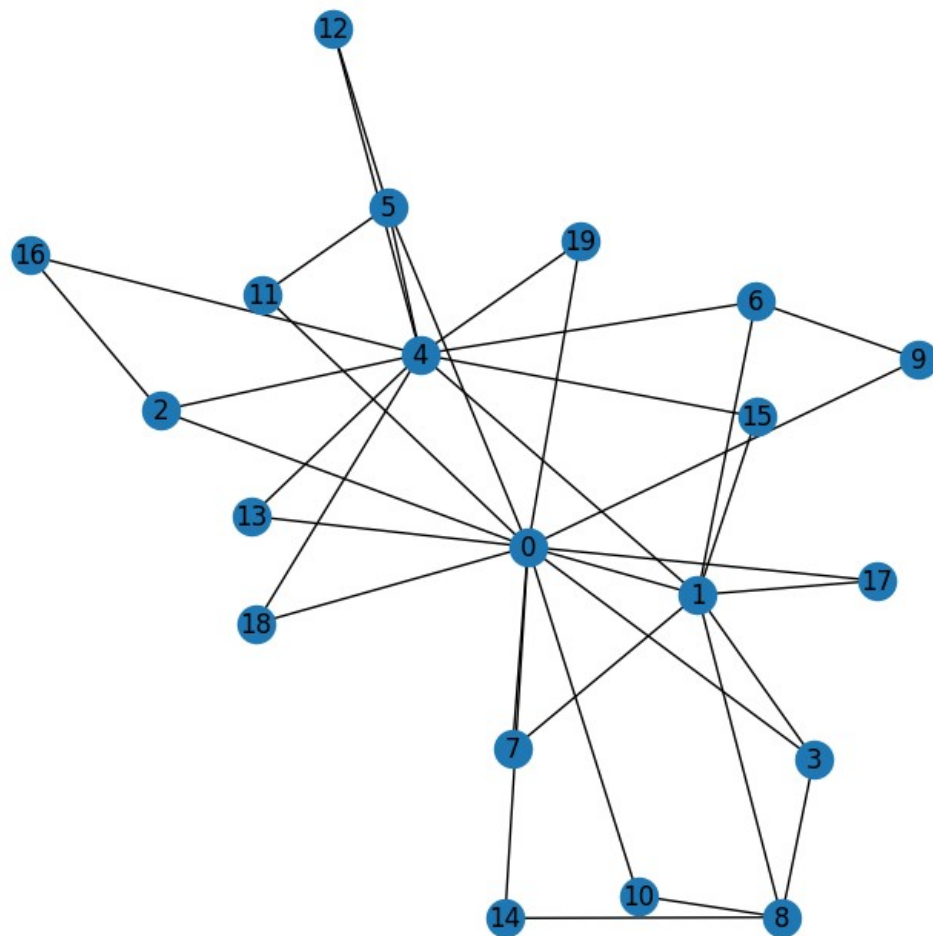
```
m0=2
```

```
G = nx.barabasi_albert_graph(N, m0, seed=42)
```

```
plt.figure(figsize=(7, 7))
```

```
pos = nx.spring_layout(G, seed=42)
```

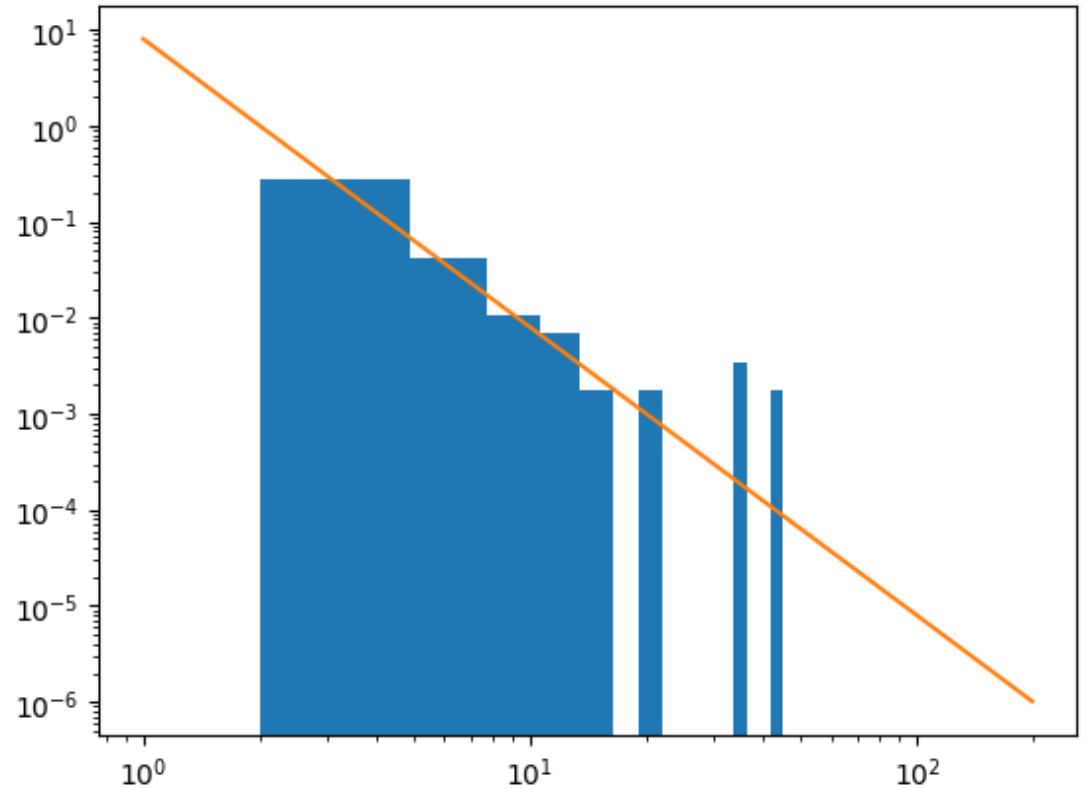
```
nx.draw(G, pos, with_labels=True)
```



```

num_nodes = G.number_of_nodes()
num_edges = G.number_of_edges()
avg_degree = sum(dict(G.degree()).values()) / num_nodes if num_nodes > 0 else 0
print("Nodes: ", num_nodes, " Edges: ", num_edges, "Avg Degree: ", avg_degree)
degrees = [d for n, d in G.degree()]
plt.hist(degrees, bins=round(np.sqrt(N))+1, density=True)
from scipy.stats import poisson
l=np.array(range(1,N)).astype(float)
plt.plot(l, 2*(m0**2)* (l**(-3)))
plt.xscale('log')
plt.yscale('log')

```



Случайные графы. Модель Уоттса-Строгаца (модель тесного мира)

Граф «Мир тесен» определяется как сеть, в которой типичное расстояние L между двумя произвольно выбранными вершинами (количество шагов, необходимых, чтобы достичь одну из другой) растёт пропорционально логарифму от числа вершин N в сети, таким образом

$$L \propto \log N$$


```
# N (int): Number of nodes  
# m0 (int): Each node is connected to k nearest neighbors (k must be even)  
# P (float): Probability of rewiring each edge ( $0 \leq p \leq 1$ )
```

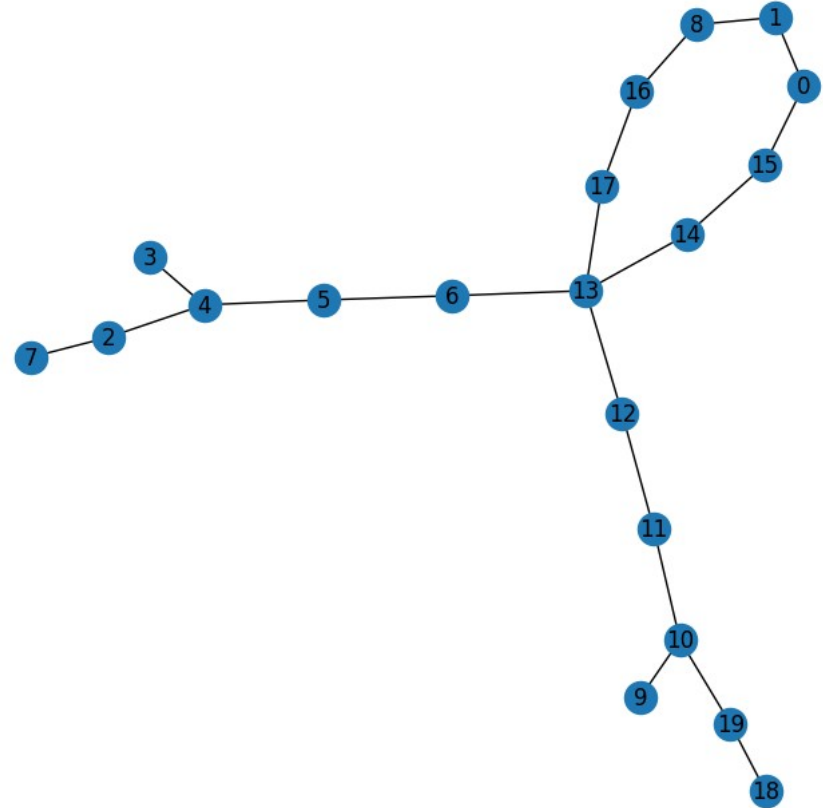
```
N=20
```

```
G = nx.watts_strogatz_graph(N, m0, P, seed=42)
```

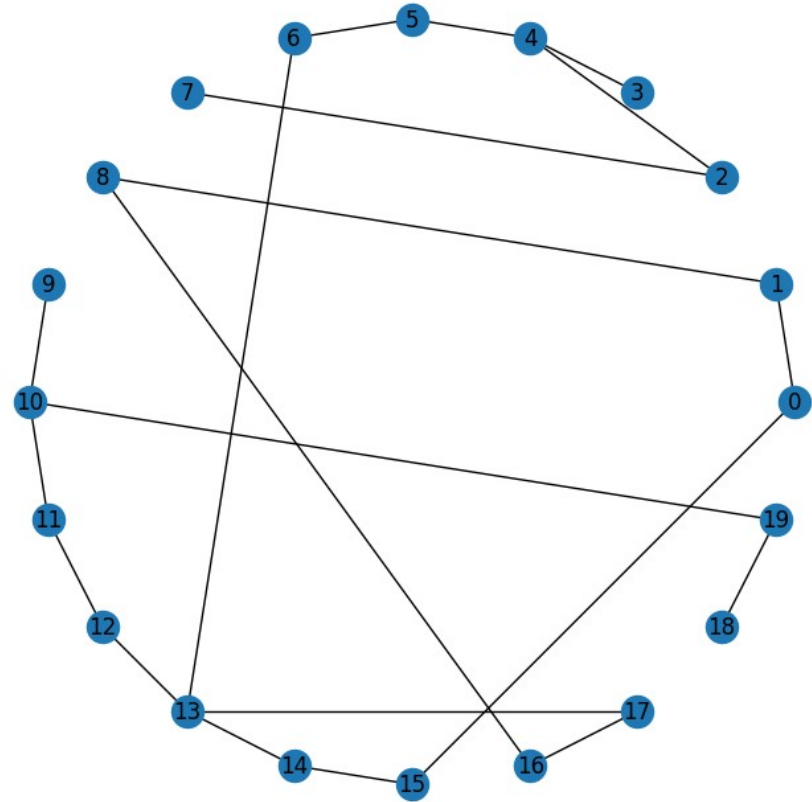
```
plt.figure(figsize=(7, 7))
```

```
pos = nx.spring_layout(G, seed=42)
```

```
nx.draw(G, pos, with_labels=True)
```



```
pos = nx.circular_layout(G)
nx.draw(G, pos, with_labels=True)
```



Алгоритм Камада-Кавай для отображения графа

d_{ij} - длина кратчайшего пути между вершинами i и j

$l_{ij} = L * d_{ij}$ - длина пружины между вершинами i и j

$k_{ij} = K/d_{ij}^2$ - сила пружины между вершинами i и j

Функция энергии

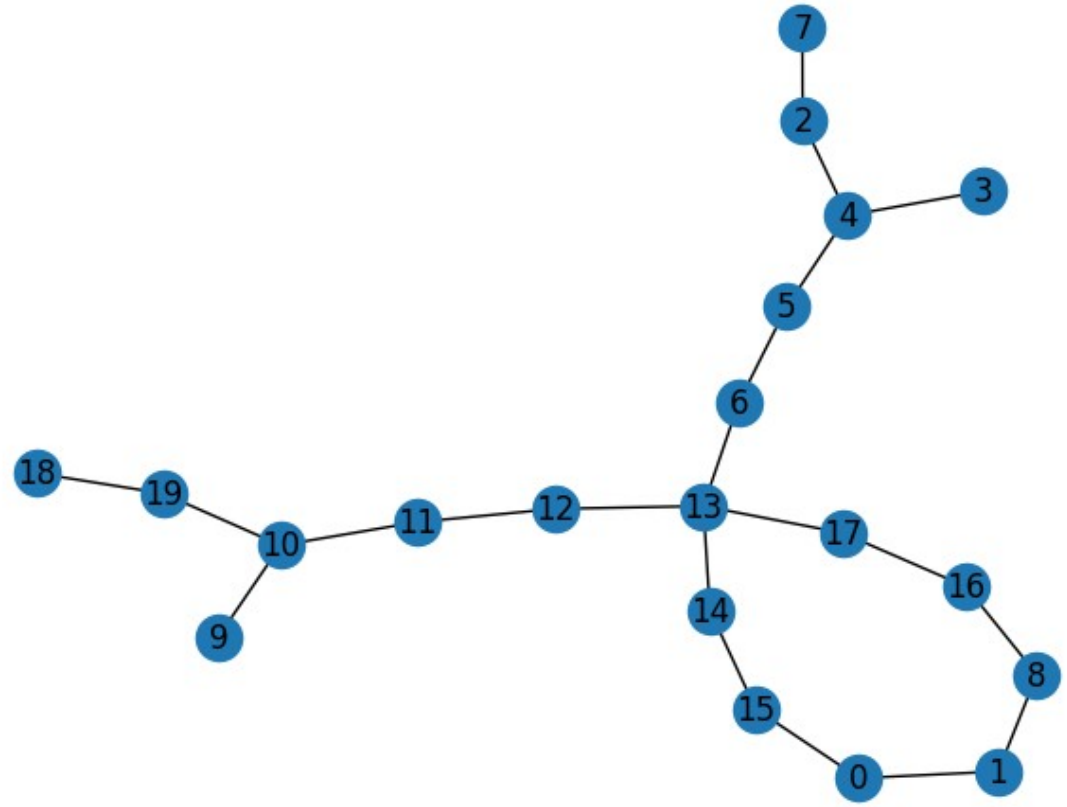
$$E_{KK} = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{1}{2} k_{ij} (||p_i - p_j|| - l_{ij})^2$$

Алгоритм является вычислительно затратным, так как для расчета кратчайших путей между вершинами требует времени $O(V^3)$

Также алгоритму требуется более 100 итераций для получения хорошего результата, что делает его неэффективным для применения на больших графах

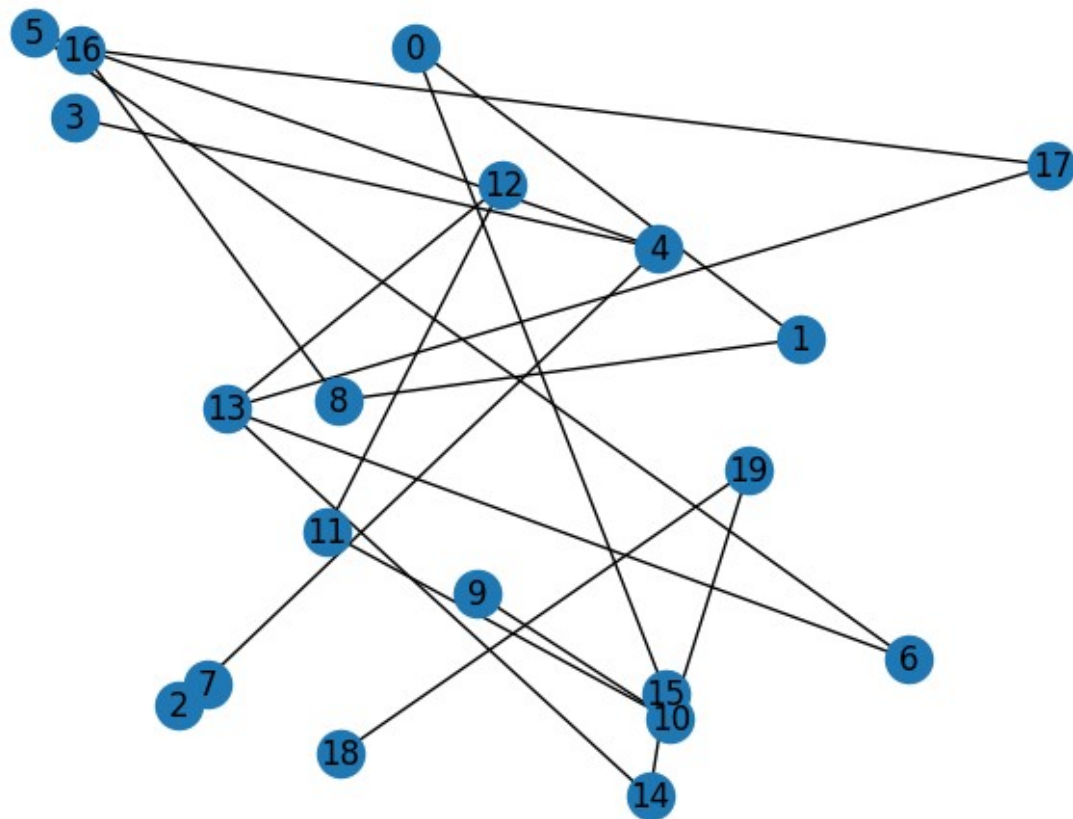
```
pos = nx.kamada_kawai_layout(G)
```

```
nx.draw(G, pos, with_labels=True)
```

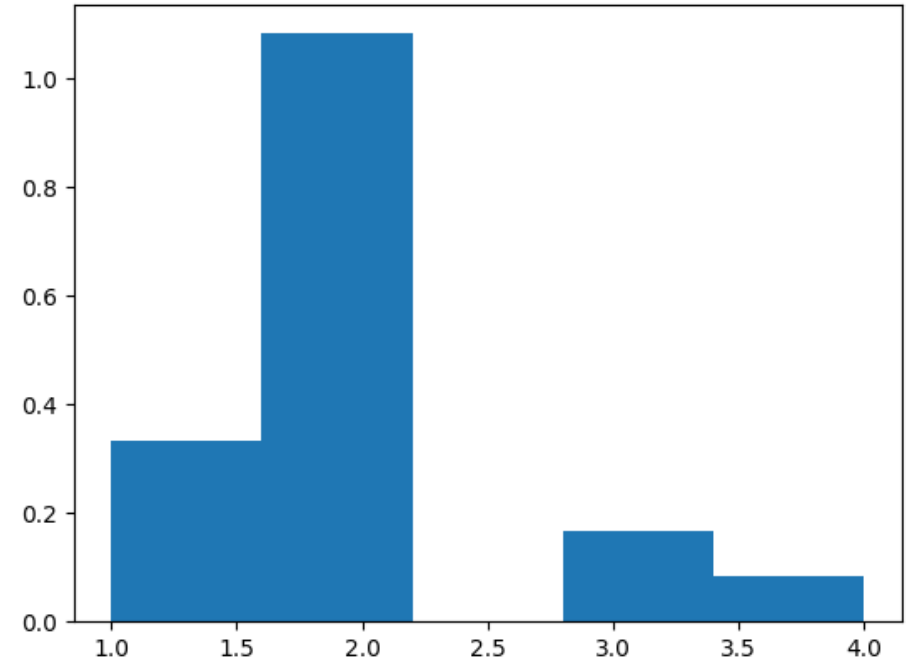


```
pos = nx.random_layout(G, seed=42)
```

```
nx.draw(G, pos, with_labels=True)
```



```
num_nodes = G.number_of_nodes()
num_edges = G.number_of_edges()
avg_degree = sum(dict(G.degree()).values()) / num_nodes if num_nodes > 0 else 0
print("Nodes: ",num_nodes," Edges: ",num_edges,"Avg Degree: ",avg_degree)
degrees = [d for n, d in G.degree()]
plt.hist(degrees,bins=round(np.sqrt(N))+1,density=True)
```



Локальный коэффициент кластеризации: доля пар друзей узла, являющихся друзьями друг друга.

Формула: количество пар друзей узла A, являющихся друзьями / количество пар друзей узла A.

Друзья – связанные ноды

Вычисление: `nx.clustering(G)`

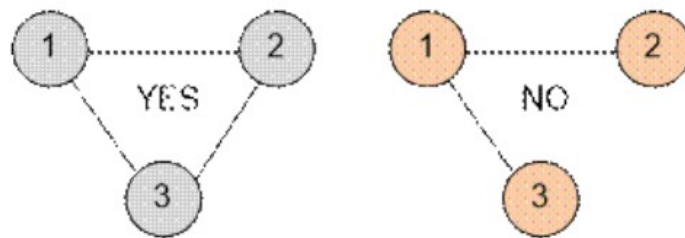
Глобальный(средний) коэффициент кластеризации рассчитывается двумя способами:

Средний локальный коэффициент кластеризации по всем узлам графа.

Вычисление: `nx.average_clustering(G)`

Транзитивность: процент «открытых триад», являющихся треугольниками в сети.

Вычисление: `nx.transitivity(G)`

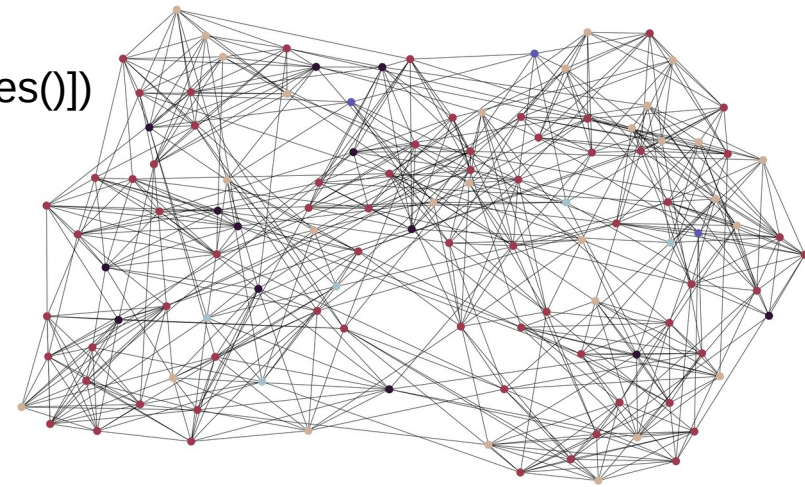


Некоторые реальные графы

```
!wget -c http://www-personal.umich.edu/~mejn/netdata/football.zip
!unzip football.zip
```

```
import matplotlib as mpl
import networkx as nx
import matplotlib.pyplot as plt
gml = open("football.gml").readlines()[1:]
G = nx.parse_gml(gml)
d = dict(G.degree)
print(set(d.values()))
low, *_ , high = sorted(d.values())
norm = mpl.colors.Normalize(vmin=low, vmax=high, clip=True)
mapper = mpl.cm.ScalarMappable(norm=norm, cmap=mpl.cm.twilight_shifted)
options = {"node_size": 50, "linewidths": 0, "width": 0.1,}
plt.figure(figsize=(30, 18))
nx.draw(G, nodelist=d, node_color=[mapper.to_rgba(i) for i in d.values()])
plt.show()
```

```
graph
[
  directed 0
  node
  [
    id 0
    label "BrighamYoung"
    value 7
  ]
  edge
  [
    source 38
    target 12
  ]
  ...
]
```



```

import numpy as np
def GetGraphMetrics(graph):
    graph_degree = dict(graph.degree)
    print("Graph Summary:")
    print(f"Number of nodes : {len(graph.nodes)}")
    print(f"Number of edges : {len(graph.edges)}")
    print(f"Maximum degree : {np.max(list(graph_degree.values()))}")
    print(f"Minimum degree : {np.min(list(graph_degree.values()))}")
    print(f"Average degree : {np.mean(list(graph_degree.values()))}")
    print(f"Median degree : {np.median(list(graph_degree.values()))}")
    print("")
    print("Graph Connectivity")
    try:
        print(f"Connected Components : {nx.number_connected_components(graph)}")
    except:
        print(f"Strongly Connected Components : {nx.number_strongly_connected_components(graph)}")
        print(f"Weakly Connected Components : {nx.number_weakly_connected_components(graph)}")
    print("")
    print("Graph Distance")
    print(f"Average Distance : {nx.average_shortest_path_length(graph)}")
    print(f"Diameter : {nx.algorithms.distance_measures.diameter(graph)}")
    print("")
    print("Graph Clustering")
    print(f"Transitivity : {nx.transitivity(graph)}")
    print(f"Average Clustering Coefficient : {nx.average_clustering(graph)}")
    return None

```

GetGraphMetrics(G)

```

import numpy as np
def GetGraphMetrics(graph):
    graph_degree = dict(graph.degree)
    print("Graph Summary:")
    print(f"Number of nodes : {len(graph.nodes)}")
    print(f"Number of edges : {len(graph.edges)}")
    print(f"Maximum degree : {np.max(list(graph_degree.values()))}")
    print(f"Minimum degree : {np.min(list(graph_degree.values()))}")
    print(f"Average degree : {np.mean(list(graph_degree.values()))}")
    print(f"Median degree : {np.median(list(graph_degree.values()))}")
    print("")
    print("Graph Connectivity")
    try:
        print(f"Connected Components : {nx.number_connected_components(graph)}")
    except:
        print(f"Strongly Connected Components : {nx.number_strongly_connected_components(graph)}")
        print(f"Weakly Connected Components : {nx.number_weakly_connected_components(graph)}")
    print("")
    print("Graph Distance")
    print(f"Average Distance : {nx.average_shortest_path_length(graph)}")
    print(f"Diameter : {nx.algorithms.distance_measures.diameter(graph)}")
    print("")
    print("Graph Clustering")
    print(f"Transitivity : {nx.transitivity(graph)}")
    print(f"Average Clustering Coefficient : {nx.average_clustering(graph)}")
    return None

```

GetGraphMetrics(G)

Graph Summary:

Number of nodes : 115

Number of edges : 613

Maximum degree : 12

Minimum degree : 7

Average degree : 10.660869565217391

Median degree : 11.0

Graph Connectivity

Connected Components : 1

Graph Distance

Average Distance : 2.5081617086193746

Diameter : 4

Graph Clustering

Transitivity : 0.4072398190045249

Average Clustering Coefficient : 0.40321601104209814

Расстояние между двумя узлами — это длина кратчайшего пути между ними. Длина пути определяется количеством шагов, которые он содержит от начала до конца, чтобы достичь узла y из узла x .

Нахождение этого расстояния, особенно для больших графов, может быть очень затратным с вычислительной точки зрения. В основе теории графов лежат два алгоритма:

Поиск в ширину (BFS): «обнаруживает» узлы в слоях на основе связности. Он начинается с корневого узла и находит все узлы в ближайшем слое связности, прежде чем продолжить обход графа.

Поиск в глубину (DFS): «посещает» узлы, проходя граф от корневого узла до его первого листового узла, прежде чем перейти к другому маршруту в графе.

Среднее расстояние: среднее расстояние между каждой парой узлов. Обычно ограничивается наибольшим компонентом, когда сеть не подключена.

Диаметр: максимальное расстояние между любой парой узлов.

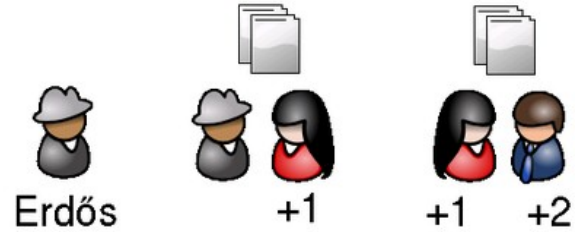
Важность ноды в графе

Центральность — это способ оценки важности узлов/рёбер в графе. В зависимости от предметной области/данных следует использовать различные предположения, что, естественно, приведёт к оценке различных мер центральности. Вот некоторые способы количественной оценки «важности» сети: степень связности, средняя близость к другим узлам, доля кратчайших путей, проходящих через узел, и т. д.

Некоторые области применения мер центральности:

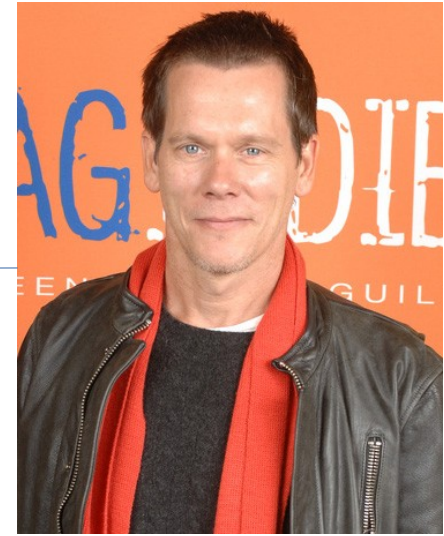
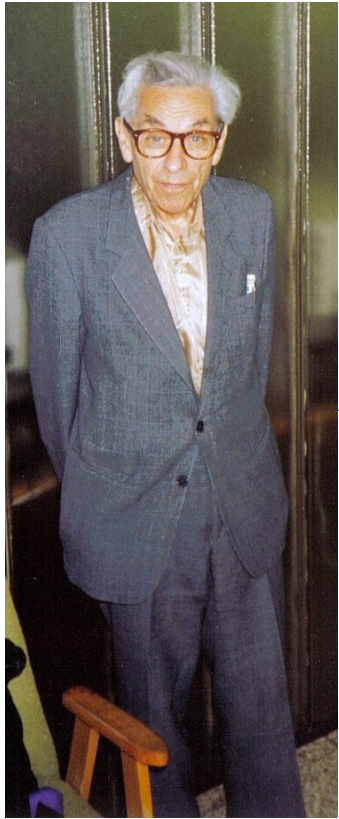
- поиск влиятельных узлов в социальной сети;
- выявление узлов, распространяющих информацию на множество узлов или предотвращающих эпидемии;
- узлы в транспортной сети;
- важные страницы в интернете;
- узлы, предотвращающие разрушение сети;
- и многое другое!

Число Эрдёша

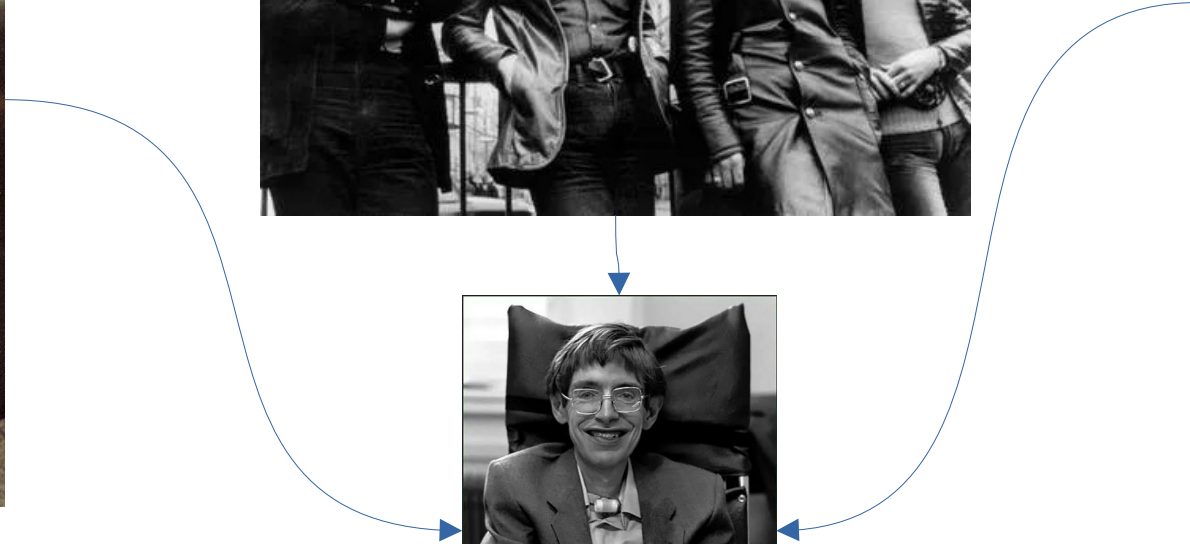
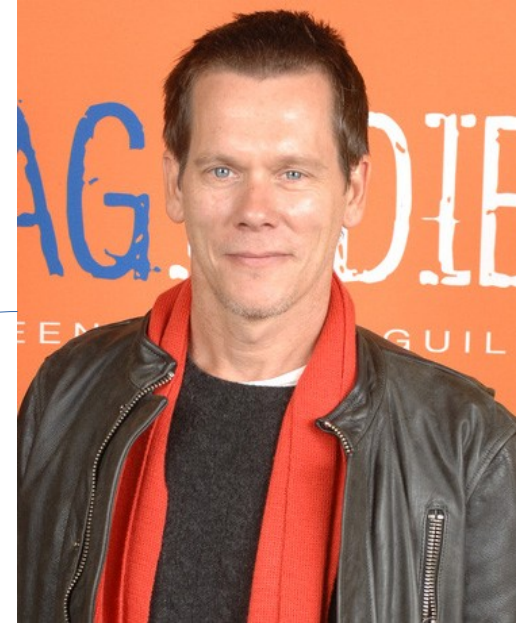
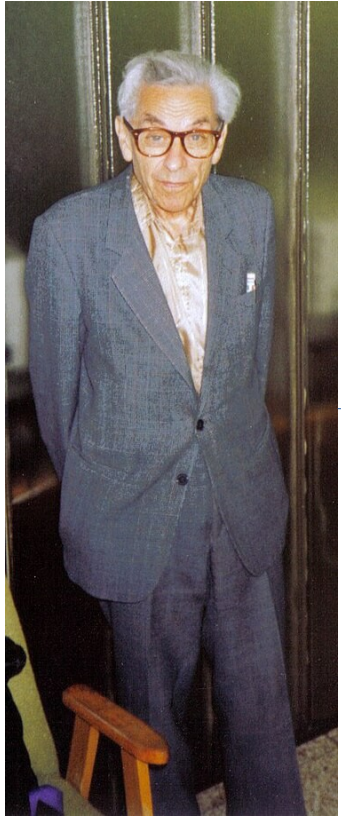


Связано как с понятием диаметра графа, так и с понятием центральности ноды в графе

Число Эрдёша — Бэйкона



Число Эрдёша-Бэйкона-Саббат



Центральность степени

Предположение: важные узлы имеют много связей.

Это самая простая мера центральности: количество соседей. Для ненаправленных сетей используется степень, а для направленных — степень входа или выхода. Значения центральности степени обычно нормализуются путем деления на максимально возможную степень в простом графе $n-1$, где n — количество узлов в графе G .

Центральность степени

Предположение: важные узлы имеют много связей.

Это самая простая мера центральности: количество соседей. Для ненаправленных сетей используется степень, а для направленных — степень входа или выхода. Значения центральности степени обычно нормализуются путем деления на максимально возможную степень в простом графе $n-1$, где n — количество узлов в графе G .

Центральность близости

Предположение: важные узлы расположены близко к другим узлам.

$$C(x) = \frac{N}{\sum_y d(y, x)}$$

Это величина, обратная среднему расстоянию кратчайшего пути до узла по всем $n-1$ достижимым узлам. Если узлы не связаны, можно либо рассматривать центральность близости, основываясь только на узлах, которые могут достичь его, либо рассматривать только узлы, которые могут достичь его, и нормализовать это значение по доле узлов, которые он может достичь.

Промежуточная центральность

Предположение: важные узлы расположены близко к другим узлам.

$$g(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

Промежуточная центральность узла v — это сумма доли всех пар кратчайших путей, проходящих через v . Формула, по сути, вычисляет количество кратчайших путей между узлами s и t , проходящих через узел v , и делит это значение на количество всех кратчайших путей между s и t (и суммирует по всем путям, которые не начинаются или не заканчиваются на v). Значения промежуточной центральности будут больше в графах с большим количеством узлов. Чтобы это контролировать, мы делим значения центральности на количество пар узлов в графе (исключая v). Сложность с этой метрикой возникает, когда в сети несколько кратчайших путей.

Поскольку вычисление этой метрики может быть очень затратным, обычно вычисляют эту метрику для выборки пар узлов. Эта метрика также может использоваться для поиска важных ребер.

Центральность по собственному вектору

Предположение: важные узлы соединены с центральными узлами.

$$Ax = \lambda x$$

Центральность по собственному вектору вычисляет центральность узла на основе центральности его соседей. Она измеряет влияние узла в сети. Относительные баллы присваиваются всем узлам сети, исходя из того, что связи с узлами с высоким баллом вносят больший вклад в оценку рассматриваемого узла, чем равные по величине баллы связи с узлами с низким баллом. Высокий балл собственного вектора означает, что узел соединен со многими узлами, которые сами имеют высокие баллы.

PageRank

Предположение: важные узлы – это узлы с большим количеством входящих ссылок из других важных узлов.

Алгоритм:

Начать со случайного узла.

Присвоить всем узлам PageRank, равный $1/n$.

Выбрать случайным образом исходящую ветвь и следовать по ней до следующего узла.

Выполнить базовое правило обновления PageRank: каждый узел отдаёт равную долю своего текущего PageRank всем узлам, на которые он ссылается.

Новый PageRank каждого узла равен сумме всех PageRank, полученных от других узлов.

Повторить k раз.

Это известный алгоритм PageRank Google. Он измеряет важность веб-страниц в структуре сети гиперссылок. Он присваивает каждому узлу оценку важности в зависимости от количества ссылок, поступающих с других узлов. В больших сетях предпочтительнее масштабированный PageRank, поскольку он имеет «параметр демпфирования» альфа. Это вероятность того, что исходящее ребро будет выбрано случайным образом для перехода к другому узлу в алгоритме, что особенно полезно, когда в сети имеется замкнутый цикл исходящих узлов.

Хабы и авторитетные узлы (HITS)

Предположение: важные узлы, имеющие входящие ребра от хороших хабов, являются хорошими авторитетными узлами, а узлы, имеющие исходящие ребра к хорошим авторитетным узлам, являются хорошими хабами.

Алгоритм:

Присвойте каждому узлу рейтинг авторитетности и хаба, равный 1.

Примените правило обновления авторитетности: рейтинг авторитетности каждого узла равен сумме рейтингов хабов каждого узла, на который он указывает.

Примените правило обновления хаба: рейтинг хаба каждого узла равен сумме рейтингов авторитетности каждого узла, на который он указывает.

Нормализуйте рейтинги авторитетности и хаба каждого узла по общему рейтингу каждого узла.

Повторите k раз.

Алгоритм HITS начинается с корня или набора высокорелевантных узлов (потенциальных авторитетных узлов). Затем все узлы, связанные с узлом в корне, являются потенциальными хабами. База определяется как корневые узлы и любой узел, связанный с узлом в корне.

Рассматриваются все ребра, соединяющие узлы в базовом наборе, и при этом основное внимание уделяется определенному подмножеству сети, имеющему отношение к конкретному запросу.

Результаты оценки центральности по разным признакам

```
import pandas as pd
def createBarChartForCentralityMeasures(sorted_centrality_teams, centrality_name):
    return pd.DataFrame(sorted_centrality_teams, columns=['Team', centrality_name])
metric_main_df = pd.DataFrame(index=list(d.keys()))
degree_centrality_df = createBarChartForCentralityMeasures(sorted(nx.degree_centrality(G).items(), key=lambda x:x[1], reverse=True), 'Degree Centrality')
closeness_centrality_df = createBarChartForCentralityMeasures(sorted(nx.closeness_centrality(G).items(), key=lambda x:x[1], reverse=True), 'Closeness Centrality')
betweenness_centrality_df = createBarChartForCentralityMeasures(sorted(nx.betweenness_centrality(G).items(), key=lambda x:x[1], reverse=True), 'Betweenness Centrality')
eigenvector_centrality_df = createBarChartForCentralityMeasures(sorted(nx.eigenvector_centrality(G).items(), key=lambda x:x[1], reverse=True), 'Eigenvector Centrality')
pagerank_df = createBarChartForCentralityMeasures(sorted(nx.pagerank(G).items(), key=lambda x:x[1], reverse=True), 'PageRank')
hub, auth = nx.hits(G)
hubs_df = createBarChartForCentralityMeasures(sorted(hub.items(), key=lambda x:x[1], reverse=True), 'Hubs')
authorities_df = createBarChartForCentralityMeasures(sorted(auth.items(), key=lambda x:x[1], reverse=True), 'Authorities')

for metric in [degree_centrality_df, closeness_centrality_df, betweenness_centrality_df, eigenvector_centrality_df, pagerank_df, hubs_df, authorities_df]:
    metric = metric.set_index('Team')
    metric_main_df = metric_main_df.join(metric)

metric_main_df
```

Результаты оценки центральности по разным признакам

	Degree Centrality	Closeness Centrality	Betweenness Centrality	Eigenvector Centrality	PageRank	Hubs	Authorities
BrighamYoung	0.105263		0.423792	0.032490	0.106503	0.009588	0.010092
FloridaState	0.105263		0.413043	0.017621	0.096385	0.009641	0.009132
Iowa	0.105263		0.407143	0.013122	0.116262	0.009509	0.011017
KansasState	0.105263		0.420664	0.023070	0.106250	0.009617	0.010069
NewMexico	0.096491		0.402827	0.010664	0.101190	0.008887	0.009589
...	...		...	...	...	...	...
TexasChristian	0.096491		0.413043	0.014370	0.114943	0.008777	0.010893
California	0.096491		0.382550	0.007516	0.111198	0.008809	0.010538
AlabamaBirmingham	0.087719		0.395833	0.011582	0.073015	0.008369	0.006918
Arkansas	0.087719		0.377483	0.006498	0.070452	0.008369	0.006675

Отображение узлов по центральности потока

```
for n in G.nodes():
    G.nodes[n]['name'] = n
    G.nodes[n]['degree'] = d[n]
    G.nodes[n]['degree_centrality'] = metric_main_df[metric_main_df.index==n]['Degree Centrality'].values[0]
    G.nodes[n]['closeness_centrality'] = metric_main_df[metric_main_df.index==n]['Closeness Centrality'].values[0]
    G.nodes[n]['betweenness_centrality'] = metric_main_df[metric_main_df.index==n]['Betweenness Centrality'].values[0]
    G.nodes[n]['log_flow_centrality'] = metric_main_df[metric_main_df.index==n]['Log Flow Centrality'].values[0]
    G.nodes[n]['flow_centrality'] = metric_main_df[metric_main_df.index==n]['Normalized Flow Centrality'].values[0]
    G.nodes[n]['eigenvector_centrality'] = metric_main_df[metric_main_df.index==n]['Eigenvector Centrality'].values[0]
    G.nodes[n]['pagerank'] = metric_main_df[metric_main_df.index==n]['PageRank'].values[0]
```

```
pos = nx.kamada_kawai_layout(G)
```

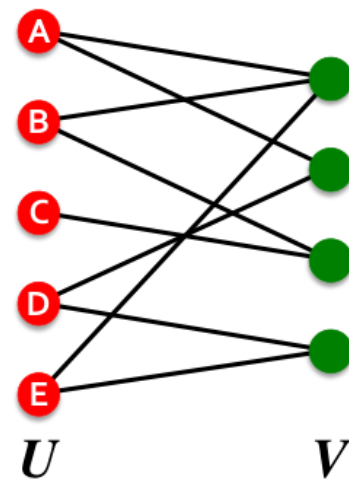
```
e = nxa.draw_networkx_edges(G, pos=pos) # get the edge layer
n = nxa.draw_networkx_nodes(G, pos=pos) # get the node layer
```

```
n = n.mark_circle().encode(
    color=alt.Color('flow_centrality:Q', scale=alt.Scale(scheme='inferno')),
    size=alt.Size('flow_centrality:Q',
        scale=alt.Scale(range=[10,100])),
    tooltip=[alt.Tooltip('name'), alt.Tooltip('flow_centrality')]
).interactive()
e = e.mark_line().encode(
    color=alt.Color('log_norm_eth',
        legend=None),
    stroke=alt.Stroke('log_norm_eth')
)
```

```
(e+n).properties(width=1500,height=800, title='CryptoPunks Network by Flow Centrality')
```

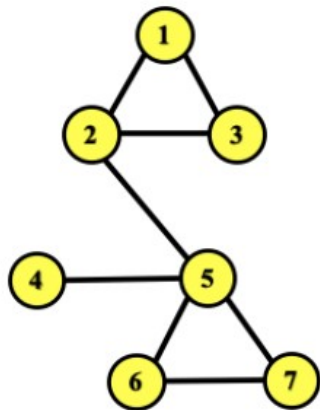
Двудольные графы

- **Bipartite graph** is a graph whose nodes can be divided into two disjoint sets U and V such that every link connects a node in U to one in V ; that is, U and V are **independent sets**
- **Examples:**
 - Authors-to-Papers (they authored)
 - Actors-to-Movies (they appeared in)
 - Users-to-Movies (they rated)
 - Recipes-to-Ingredients (they contain)
- **“Folded” networks:**
 - Author collaboration networks
 - Movie co-rating networks

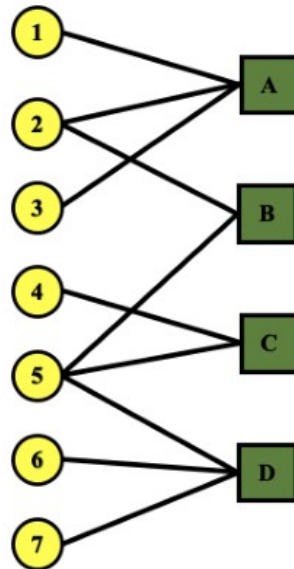


Двудольные графы

Projection U

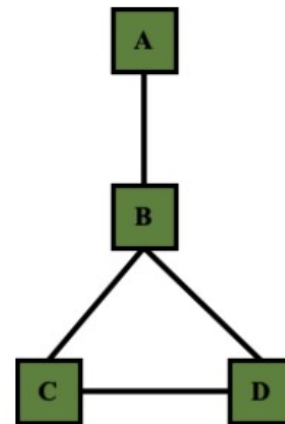


U



V

Projection V



Плотность матрицы L/N^2 или $k/(N-1)$.
 k – степень вершины

Если $k \ll N-1$ то матрица разреженная

Большинство графов реального мира - разреженные

NETWORK	NODES	LINKS	DIRECTED/ UNDIRECTED	N	L	<k>
Internet	Routers	Internet connections	Undirected	192,244	609,066	6.33
WWW	Webpages	Links	Directed	325,729	1,497,134	4.60
Power Grid	Power plants, transformers	Cables	Undirected	4,941	6,594	2.67
Phone Calls	Subscribers	Calls	Directed	36,595	91,826	2.51
Email	Email Addresses	Emails	Directed	57,194	103,731	1.81
Science Collaboration	Scientists	Co-authorship	Undirected	23,133	93,439	8.08
Actor Network	Actors	Co-acting	Undirected	702,388	29,397,908	83.71
Citation Network	Paper	Citations	Directed	449,673	4,689,479	10.43
E. Coli Metabolism	Metabolites	Chemical reactions	Directed	1,039	5,802	5.58
Protein Interactions	Proteins	Binding interactions	Undirected	2,018	2,930	2.90